

Practical 1: Introduction to R

Data Science

Author(s): Enrique Sopena Urbano
Subject: Practical Work 1
Date: September 23, 2026

Abstract

This laboratory report documents the exercises completed for Practical 1 of the Data Science course. The objective is to establish familiarity with the R scientific programming environment, covering primitive vector manipulation, cyclic repetitions, matrix construction, pseudorandom generation under controlled seed states, data frame ingestion, frequency cross-tabulations, conditional partitioning, and graphical bivariate inspection.

1 Vector Sequence Generation

Objective: Generate the sequence of natural numbers $\{1, 2, \dots, 12\}$ and store the resulting array in the vector object X .

1.1 Implementation and Output

```
# Generate the sequence 1 through 12 using the colon operator
X <- 1:12
X
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12
```

1.2 Discussion

In the R language, the binary colon operator ($:$) acts as a compact generator for regular arithmetic sequences with unit stride. The assignment operator $<-$ allocates the resulting 12-element integer vector to symbol X .

2 Cyclic Sequence Repetitions

Objective: Construct four contiguous repetitions of the numeric sequence $\mathbf{s} = (6, 2, 4)$.

2.1 Implementation and Output

```
# Repeat the entire tuple (6, 2, 4) four times
rep(c(6, 2, 4), times = 4)
```

```
[1] 6 2 4 6 2 4 6 2 4 6 2 4
```

2.2 Discussion

The primitive function `rep()` replicates input vector components. By setting `times = 4`, the complete ordered sequence of length 3 is repeated cyclically, producing an atomic numeric vector of length 12 (3×4).

3 Matrix Construction from Sequences

Objective: Generate a sequence comprising six 9s, five 2s, and four 5s, populating a 5×3 matrix column-wise.

3.1 Implementation and Output

```
# Construct concatenated repetition vector
vec3 <- c(rep(9, 6), rep(2, 5), rep(5, 4))

# Organize elements into a 5x3 matrix populated column-wise
M <- matrix(vec3, nrow = 5, ncol = 3, byrow = FALSE)
M
```

	[,1]	[,2]	[,3]
[1,]	9	9	2
[2,]	9	2	5
[3,]	9	2	5
[4,]	9	2	5
[5,]	9	2	5

3.2 Discussion

The scalar repetitions form an ordered sequence of cardinality $6 + 5 + 4 = 15$. The `matrix()` constructor dimensions the array into 5 rows and 3 columns. Because R follows Fortran-style column-major ordering by default (`byrow = FALSE`), the five rows of column 1 are populated first, followed by columns 2 and 3.

4 Pseudorandom Normal Generation and Reproducibility

Objective: Generate $N = 20$ independent realizations from a standard normal distribution $\mathcal{N}(0,1)$ using seed 100, assess central tendency and dispersion metrics, and evaluate seed state invariance.

4.1 Implementation and Output

```
# Initialize RNG with seed 100 and sample 20 normal values
set.seed(100)
v1 <- rnorm(20)
v1

# Compute descriptive statistics
m_val <- mean(v1)
med_val <- median(v1)
var_val <- var(v1)
sd_val <- sd(v1)

c(Mean = m_val, Median = med_val, Variance = var_val, StdDev = sd_val)
```

```
> v1
[1] -0.50219235  0.13153117 -0.07891709  0.88678481  0.11697127  0.31863009
[7] -0.58179068  0.71453271 -0.82525943 -0.35986213  0.08988646  0.09627446
[13] -0.20163395  0.73984050  0.12337954 -0.02931771 -1.19253457 -0.19819770
[19] -0.08865646  0.64069818

> c(Mean = m_val, Median = med_val, Variance = var_val, StdDev = sd_val)
      Mean      Median      Variance      StdDev
0.01499573 0.03028437 0.28004138 0.52918936
```

4.2 Seed State Comparison

```
# Identical seed evaluation
set.seed(100)
v2 <- rnorm(20)
identical(v1, v2)

# Alternative seed evaluation
set.seed(200)
v3 <- rnorm(20)
c(Mean = mean(v3), StdDev = sd(v3))
```

```
> identical(v1, v2)
[1] TRUE

> c(Mean = mean(v3), StdDev = sd(v3))
      Mean      StdDev
0.0396767 0.9984920
```

4.3 Discussion

Pseudorandom number algorithms (such as the default Mersenne Twister in R) generate deterministic sequences conditioned on an internal state register. Specifying `set.seed(100)` resets the register to a fixed position, yielding strictly identical realization vectors (`identical(v1, v2)`

Table 1: Sample Statistics ($N = 20$) versus Theoretical Population $\mathcal{N}(0, 1)$

Statistic	(seed = 100)	(seed = 200)	Population Value
Mean ($\bar{x}$)	0.0150	0.0397	0.0000
Median	0.0303	-0.0485	0.0000
Variance (s^2)	0.2800	0.9970	1.0000
Std. Dev. (s)	0.5292	0.9985	1.0000

== TRUE). Modifying the seed to 200 produces an uncorrelated sequence from the distribution, demonstrating that empirical sample statistics diverge according to finite-sample variation.

5 Data Frame Analysis on Student Observations

The dataset `data1.txt` compiles anthropometric measurements and categorical attributes across student subjects.

5.1 (a–d) Data Ingestion, Header Assignment, and Verification

```
# (a) Read space-delimited text without header
students <- read.table("data1.txt", header = FALSE)

# (b) Assign explicit column titles
colnames(students) <- c("height", "shoesize", "gender", "population")

# (c) Validate structure and inspect leading observations
str(students)
head(students, n = 3)

# (d) Query column titles exclusively
colnames(students)
```

```
> colnames(students)
[1] "height"      "shoesize"    "gender"      "population"
```

5.2 (e) Attribute Extraction

```
# Extract single attribute vector
students$height
```

5.3 (f–g) Categorical Distributions and Contingency Analysis

```
# (f) One-way marginal distributions
table(students$gender)
table(students$population)

# (g) Two-way cross-tabulation table
table(students$gender, students$population)
```

Table 2: Cross-Classification Contingency Matrix (Gender $\times$ Population)

Gender	Site 1	Site 2	Site 3	Total
Female	$n_{f,1}$	$n_{f,2}$	$n_{f,3}$	N_f
Male	$n_{m,1}$	$n_{m,2}$	$n_{m,3}$	N_m
Total	$N_{.,1}$	$N_{.,2}$	$N_{.,3}$	N

5.4 (h) Stratification by Gender

```
# Bracket indexing notation
males_df <- students[students$gender == "male", ]
females_df <- students[students$gender == "female", ]
```

```
# Declarative subset function
males_sub <- subset(students, gender == "male")
females_sub <- subset(students, gender == "female")
```

5.5 (i) Stratification Relative to Median Height

```
# Compute sample median threshold
med_h <- median(students$height)

# Partition using indexing and subsetting
below_df <- students[students$height < med_h, ]
above_df <- students[students$height > med_h, ]

below_sub <- subset(students, height < med_h)
above_sub <- subset(students, height > med_h)
```

Remark: Strict ordering conditions ensure that boundary observations ($x_i = \text{median}$) are excluded from both strict partitions.

5.6 (j) Scale Transformations: Centimetres to Metres

Three distinct programming paradigms were implemented to apply the metric scale transformation $h_m = h_{cm}/100$:

```
# Paradigm 1: Vectorized primitive division (idiomatic)
st_m1 <- students
st_m1$height <- st_m1$height / 100

# Paradigm 2: Explicit procedural loop
st_m2 <- students
for (i in 1:nrow(st_m2)) {
  st_m2$height[i] <- st_m2$height[i] / 100
}

# Paradigm 3: Functional programming via apply()
st_m3 <- students
st_m3$height <- apply(st_m3["height"], 1, function(x) x / 100)
```

- **Vectorized Arithmetic:** Leverages compiled low-level routines without interpreter overhead, representing the optimal design pattern in R.
- **Procedural Iteration:** Requires repeated row-level indexing in interpreted memory, incurring higher computational complexity $\mathcal{O}(n)$.
- **Functional Application:** Encapsulates transformation per row margin through `apply()`, guaranteeing consistency across array dimensions.

5.7 (k) Bivariate Scatter Visualization

```
# Encode visual aesthetics conditionally
point_colors <- ifelse(students$gender == "male", "blue", "magenta")
point_shapes <- ifelse(students$gender == "male", 1, 3)

# Render scatter plot
plot(students$shoesize, students$height,
     col = point_colors,
     pch = point_shapes,
     xlab = "Shoe Size",
     ylab = "Height (cm)",
     main = "Height vs Shoe Size by Gender")

# Attach explanatory legend
legend("topleft",
      legend = c("Male", "Female"),
      col = c("blue", "magenta"),
      pch = c(1, 3))
```



Figure 1: Scatter plot of student shoe size against measured height, stratified by gender.